

Two Ways to Recommend a Song

Content-based vs. collaborative filtering, built and compared head to head

Your Name

2026-09-24

Table of contents

1	Introduction	2
2	Part I — Content-Based Recommendation (Spotify)	2
2.1	Data	2
2.1.1	Why popularity and track_genre are excluded from the feature space	3
2.2	Feature engineering	3
2.3	Methodology	5
2.3.1	The recommender: weighted nearest-neighbor search	5
2.3.2	Evaluating a recommender with no user-feedback data: precision@k	5
2.3.3	Is exact-genre matching too strict?	7
2.4	Interpretability: what separates genres?	7
3	Part II — Collaborative Filtering (KKBox)	8
3.1	Data	8
3.1.1	Real data-quality issues, found by checking rather than assuming	9
3.2	Feature engineering	10
3.3	Methodology	11
3.3.1	Why one train/holdout split, not k-fold cross-validation	11
3.3.2	Model comparison	11
3.3.3	Full-scale result	12
3.4	Interpretability	12
4	Head-to-Head Comparison	13
5	Discussion	15
6	Conclusion	15

1 Introduction

Most Kaggle Learn-style tabular projects — including this portfolio’s `academic_success` and `CO2 Emissions` projects — are supervised: there’s a `y` the model is trying to predict, and a metric that scores how close its predictions come. **Recommendation** is a different kind of problem, and it splits into two genuinely different techniques depending on what data is available:

- **Content-based** — no user data needed at all. Compare items by their own measurable properties (here, Spotify’s audio features) and recommend whatever’s most similar.
- **Collaborative filtering** — no item content needed at all. Learn from real user/item interaction history (here, KKBox’s listening logs) instead.

This report builds and evaluates one working system of each kind, then puts them side by side — not because one technique “wins” in general (they solve different problems, with different data requirements), but because comparing how each is built, evaluated, and interpreted is a more honest way to understand recommendation than presenting either one in isolation.

2 Part I — Content-Based Recommendation (Spotify)

Given a track someone likes, recommend other tracks whose *audio* resembles it — not tracks other, similar listeners happened to play. Every recommendation here comes from Spotify’s own audio-feature measurements: danceability, energy, acousticness, and so on. Built on Kaggle’s [Spotify Tracks Dataset](#).

2.1 Data

~114,000 tracks, roughly 1,000 sampled per genre across 114 genres, each with:

- **Audio features** (Spotify’s own per-track measurements): `danceability`, `energy`, `loudness`, `speechiness`, `acousticness`, `instrumentalness`, `liveness`, `valence` (musical positivity), `tempo`, `key` (one of 12 pitch classes), `mode` (major/minor), `time_signature`.
- **Metadata**: `track_name`, `artists`, `album_name`, `duration_ms`, `explicit`.
- **popularity** — a Spotify-computed score, and **track_genre** — a human-assigned genre tag. Neither is used as a recommender *input* feature (see Methodology) — both are reserved for evaluation.

```
from recommendation_showcase.content_based import config, data

tracks_df = data.load_tracks()
print(f"{len(tracks_df)} tracks, {tracks_df[config.GENRE_COL].nunique()} genres")
tracks_df[config.RAW_AUDIO_NUMERIC_COLS + [config.GENRE_COL, config.POPULARITY_COL]].describe()
```

89741 tracks, 113 genres

	count	unique	top	freq	mean	std	min	25%	50%
danceability	89741.0	NaN	NaN	NaN	0.562166	0.176691	0.0	0.45	0.576
energy	89741.0	NaN	NaN	NaN	0.634458	0.256605	0.0	0.457	0.676
loudness	89741.0	NaN	NaN	NaN	-8.499004	5.22149	-49.531	-10.322	-7.185
speechiness	89741.0	NaN	NaN	NaN	0.087442	0.113277	0.0	0.036	0.0489
acousticness	89741.0	NaN	NaN	NaN	0.328289	0.338321	0.0	0.0171	0.188
instrumentalness	89741.0	NaN	NaN	NaN	0.173413	0.323848	0.0	0.0	0.000058
liveness	89741.0	NaN	NaN	NaN	0.21697	0.194884	0.0	0.0982	0.132
valence	89741.0	NaN	NaN	NaN	0.469477	0.262864	0.0	0.249	0.457
tempo	89741.0	NaN	NaN	NaN	122.058316	30.117532	0.0	99.264	122.013
track_genre	89741	113	acoustic	1000	NaN	NaN	NaN	NaN	NaN
popularity	89741.0	NaN	NaN	NaN	33.198438	20.580824	0.0	19.0	33.0

2.1.1 Why popularity and track_genre are excluded from the feature space

Including `popularity` as a recommendation feature would make the system chase what’s already trending rather than what actually sounds similar — the opposite of a useful recommender. Including `track_genre` directly would make genre-matching in the evaluation below trivially easy (the recommender would just be looking up the genre column), which would prove nothing about whether the *audio* similarity is meaningful. Both are kept out of the feature space and used only afterward, to score results.

2.2 Feature engineering

Three engineered features, detailed in `src/recommendation_showcase/content_based/features.py`:

- **key_sin / key_cos** — `key` is 12 pitch classes arranged in a circle (key 11 is one semitone from key 0, not “far away” from it), so it’s encoded as a point on a circle rather than a plain integer or a flat one-hot encoding — the same technique the CO2 Emissions project uses for week-of-year.
- **duration_min** — `duration_ms` / 60000, a pure readability transform.

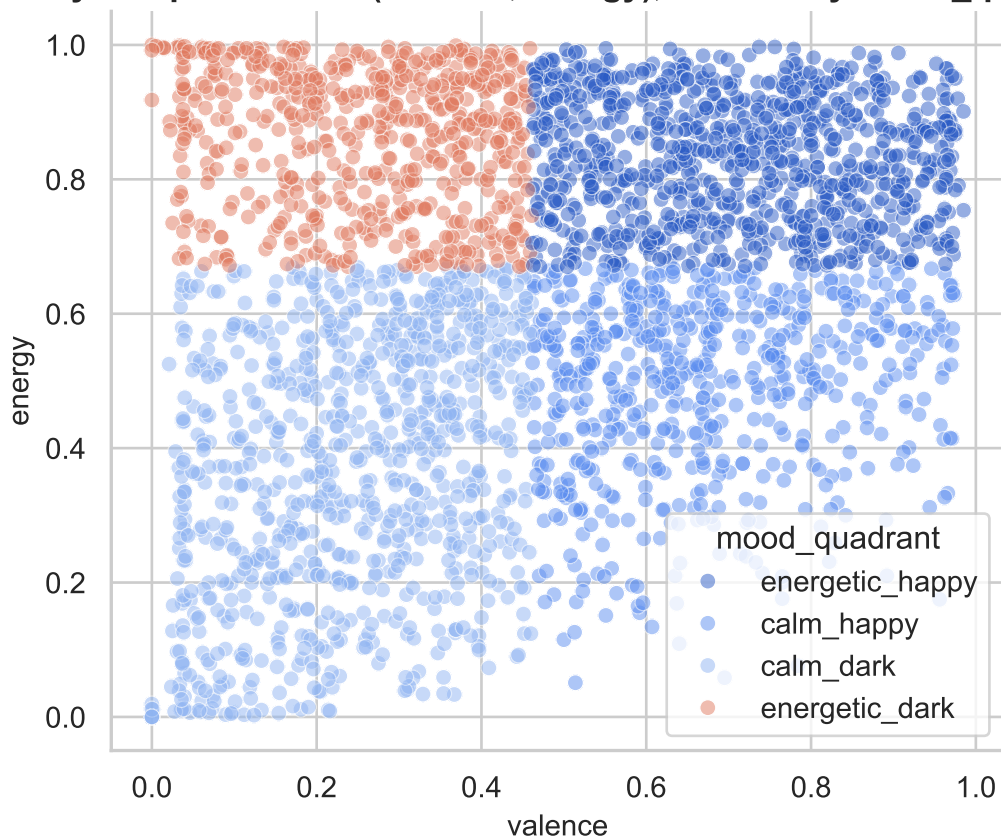
- **mood_quadrant** — buckets each track into one of four moods (energetic/calm x happy/dark) from Russell’s circumplex model of affect, using training-data medians of **valence/energy** as the quadrant boundaries. A classifier can already learn this distinction from the raw floats; this feature exists for human legibility in the notebook and app.

```
from recommendation_showcase.content_based.features import FeatureEngineer

sample = tracks_df.sample(3000, random_state=config.RANDOM_SEED)
engineered = FeatureEngineer().fit_transform(sample)

fig, ax = plt.subplots(figsize=(6, 5))
sns.scatterplot(data=engineered, x="valence", y="energy", hue="mood_quadrant", alpha=0.5, ax=ax)
ax.set_title("Every sampled track's (valence, energy), colored by mood_quadrant")
plt.show()
```

Every sampled track's (valence, energy), colored by mood_quadrant



2.3 Methodology

2.3.1 The recommender: weighted nearest-neighbor search

`ContentRecommender` (`content_based/model.py`) standardizes every audio feature (scale matters directly for distance, unlike a tree-based model), then finds the k nearest tracks by Euclidean distance in that feature space. Three optional ways to reshape that distance metric before the search runs, each evaluated below rather than assumed to help:

- **Importance-weighted** — fit a genre classifier on the same features and use its learned feature importances to *weight* the distance metric.
- **PCA** — project onto the top 10 principal components before searching. With only ~25 features to begin with, this is more a test of whether discarding lower-variance components denoises the space than a dimensionality-curse fix.
- **NCA** (Neighborhood Components Analysis) — learn a linear projection via gradient descent that directly maximizes KNN accuracy under the genre labels. Its cost scales roughly quadratically with row count and gets OOM-killed on this project’s full ~72k-row training split (verified directly), so it’s fit on a 5,000-row subsample instead.

2.3.2 Evaluating a recommender with no user-feedback data: precision@k

There’s no click-through or replay data here, so this project uses **precision@k**: hold out a set of tracks, recommend k tracks for each from the remaining catalog, and measure what fraction of those recommendations share the query track’s true genre — a rough proxy for “actually sounds alike,” the standard fallback when no real feedback signal exists.

```
from recommendation_showcase.content_based import model

train_df, test_df = model.train_test_split_tracks(tracks_df, test_size=0.2)

X_train, y_train = data.split_features_target(train_df)
classifier_pipeline = model.build_classifier_pipeline(model.random_forest_estimator())
classifier_pipeline.fit(X_train, y_train)
weights = model.compute_feature_weights_from_classifier(classifier_pipeline)

results = pd.DataFrame([
    {"approach": "Popularity baseline", "precision_at_10": model.popularity_baseline_precision_at_k(train_df, test_df, k=10)},
    {"approach": "Unweighted KNN", "precision_at_10": model.precision_at_k(train_df, test_df, k=10)},
    {"approach": "Importance-weighted KNN", "precision_at_10": model.precision_at_k(train_df, test_df, k=10, weights=weights)},
    {"approach": "PCA(10) KNN", "precision_at_10": model.precision_at_k(train_df, test_df, k=10, weights=weights)},
    {"approach": "NCA(10) KNN", "precision_at_10": model.precision_at_k(train_df, test_df, k=10, weights=weights)}
```

```

    "precision_at_10": model.precision_at_k(
        train_df, test_df, k=10, embedding=model.nca_embedding(10), embedding_fit_sample=
    ),
},
])
results

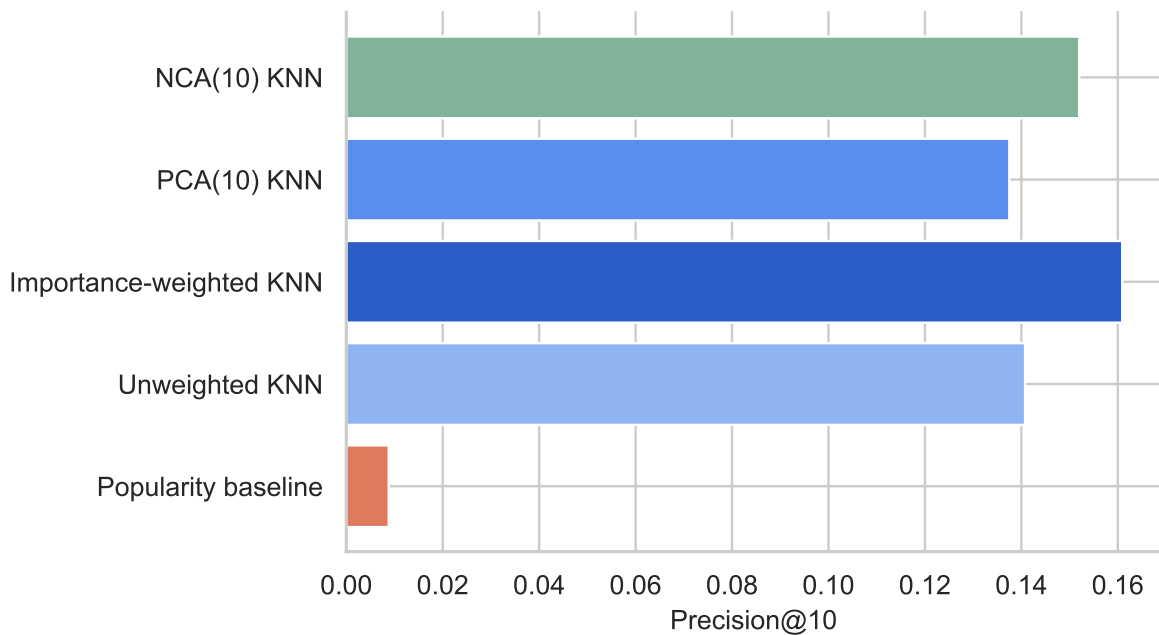
```

	approach	precision_at_10
0	Popularity baseline	0.008808
1	Unweighted KNN	0.140771
2	Importance-weighted KNN	0.160911
3	PCA(10) KNN	0.137506
4	NCA(10) KNN	0.152003

```

fig, ax = plt.subplots(figsize=(6, 4))
ax.barh(results["approach"], results["precision_at_10"], color=[PALETTE[3], PALETTE[2], PALETTE[1], PALETTE[0], PALETTE[4]])
ax.set_xlabel("Precision@10")
plt.show()

```



Importance-weighted KNN was the best of the five — the more “principled” methods (PCA, NCA) didn’t beat this simpler heuristic here, a result only knowable by measuring, not by assuming a fancier-sounding technique wins.

2.3.3 Is exact-genre matching too strict?

Several of this dataset's 113 genre tags are near-synonyms applied to the *same physical track* — verified directly: 24,259 of the raw 114,000 rows are exact-duplicate `track_ids` carrying a different genre tag each time. `genre_families.py` groups the 113 tags into 13 broader families (plus a `mood_context` bucket for tags like `chill/sad/ study` that are playlist moods, not musical genres) to see how much of that gap is near-synonym mismatches.

```
from recommendation_showcase.content_based.genre_families import GENRE_TO_FAMILY

granularity = pd.DataFrame([
    {"approach": "Popularity baseline", "granularity": "Exact genre", "precision_at_10": mod},
    {"approach": "Popularity baseline", "granularity": "Genre family", "precision_at_10": mod},
    {"approach": "Importance-weighted KNN", "granularity": "Exact genre", "precision_at_10": mod},
    {"approach": "Importance-weighted KNN", "granularity": "Genre family", "precision_at_10": mod}
])
granularity.pivot(index="approach", columns="granularity", values="precision_at_10")
```

granularity	Exact genre	Genre family
approach		
Importance-weighted KNN	0.160911	0.331155
Popularity baseline	0.008808	0.122625

The popularity baseline jumps ~14x at the looser genre-family granularity, while the recommender only roughly doubles — so the recommender's advantage over the naive baseline is actually *larger* in relative terms at the strict exact-genre level (~18x) than at the family level (~2.7x). Coarser bins flatter every method's absolute score, including guessing.

2.4 Interpretability: what separates genres?

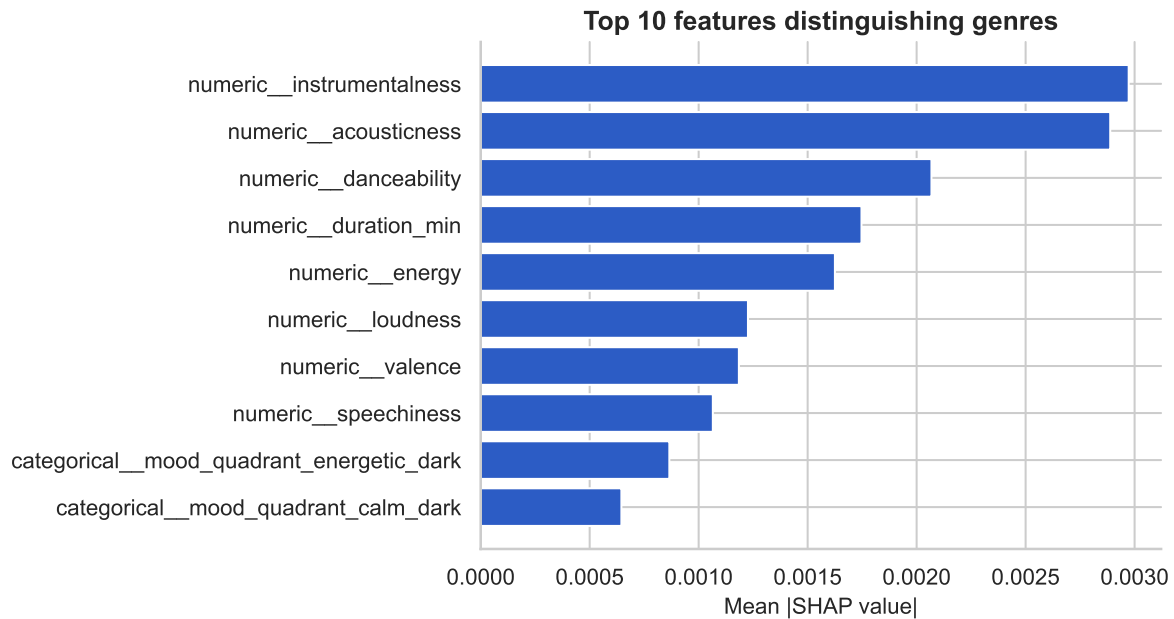
The genre classifier exists specifically to produce the importance weights above and to be explained — not to be a high-scoring classifier in its own right (114 genres is a genuinely hard multiclass problem).

```
from recommendation_showcase.content_based.interpretability import compute_shap_values, top_shap_features

explanation, _ = compute_shap_values(classifier_pipeline, X_train, max_samples=500)
top = top_shap_features(explanation, top_n=10).sort_values("mean_abs_shap")

fig, ax = plt.subplots(figsize=(6, 4.5))
```

```
ax.barh(top["feature"], top["mean_abs_shap"], color=PALETTE[0])
ax.set_xlabel("Mean |SHAP value|")
ax.set_title("Top 10 features distinguishing genres")
plt.show()
```



3 Part II — Collaborative Filtering (KKBox)

Given a real member's listening history and a specific song, predict whether they'll listen to it again within a month of first hearing it — built on Kaggle's real [WSDM — KKBox's Music Recommendation Challenge](#) (2018). Unlike Part I, this needs no audio content at all — the whole task is defined by real user/song interaction history, and there's a real Kaggle leaderboard here, scored by AUC.

3.1 Data

Four tables, verified directly against the real downloaded files:

- **train.csv/test.csv** (7,377,418 / 2,556,791 rows): one row per listening event — **msno** (member), **song_id**, three context columns (**source_system_tab**, **source_screen_name**, **source_type**), and (train only) **target**.

- **songs.csv** (2,296,834 rows): `song_length`, `genre_ids`, `artist_name`, `composer`, `lyricist`, `language`.
- **members.csv** (34,403 rows): `city`, `bd` (self-reported age), `gender`, `registered_via`, `registration_init_time`, `expiration_date`.
- **song_extra_info.csv** (2,296,870 rows): `name`, `isrc`.

```
from recommendation_showcase.collaborative import config, data

train_sample = data.load_train().sample(50_000, random_state=config.RANDOM_SEED)
panel_sample = data.merge_side_tables(train_sample)
print(f"target balance (sample): {panel_sample[config.TARGET_COL].mean():.3f}")
panel_sample.describe(include="all").T.head(10)
```

target balance (sample): 0.506

	count	unique	top	freq
msno	50000	16634	o+5RNlSWrzvrphgBNGIo1FLkGxBgyICns6qXj3nS7Pk=	46
song_id	50000	20987	reXuGcEWDDCnL0K3Th//3DFG4S1ACSpJMzA+CFipo1g=	107
source_system_tab	49815	8	my library	250
source_screen_name	47204	19	Local playlist more	218
source_type	49848	12	local-library	153
target	50000.0	NaN	NaN	Na
song_length	50000.0	NaN	NaN	Na
genre_ids	49200	220	465	251
artist_name	50000	5532	Various Artists	191
composer	38785	7969		138

Verified directly on the full 7.4M-row `train.csv`: `target` is close to balanced (50.4% replay / 49.6% no-replay) — unlike `academic_success`'s imbalanced classes, no resampling (ADASYN etc.) is needed here.

3.1.1 Real data-quality issues, found by checking rather than assuming

- **bd (age)** is garbage for ~58% of members — 0 (the unset default), negative values, and a maximum of 1051. Anything outside 5-100 becomes NaN rather than a trusted age.
- **gender missing for ~58%**, **composer missing for ~47%**, **lyricist missing for ~85%** — all three are used only as coarse presence signals, not rich per-value features.
- **genre_ids is multi-valued for ~7.9%** of songs (pipe-separated) — counted, not just checked for presence.
- **artist_name has 222,363 distinct values** — far too many to one-hot encode; replaced with a frequency encoding instead (see Methodology).

3.2 Feature engineering

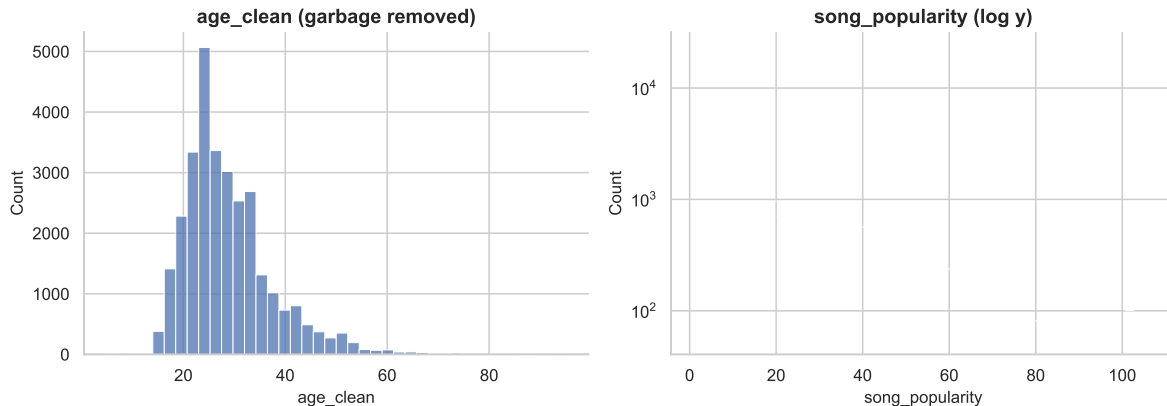
`src/recommendation_showcase/collaborative/features.py` is the single source of truth for turning these raw, messy columns into model-ready features:

- **InteractionCountEncoder:** learns `song_popularity`, `user_activity_count`, and `artist_song_count` from the *training* interactions only (a song/user/artist never seen in training gets 0 — itself meaningful signal, not a gap to fill). This is the collaborative-filtering analogue of Part I's evaluation discipline: learn a lookup from training data only, apply it unchanged to any future row.
- **FeatureEngineer:** `member_tenure_days` (registration to expiration), `age_clean` (the garbage-bd fix above), `genre_count/primary_genre` (from pipe-separated `genre_ids`), `composer_count/has_composer/has_lyricist`, and `isrc_country/isrc_year` (parsed from `song_extra_info.isrc`, with a century pivot for the 2-digit year).
- **RareCategoryGrouper:** the same technique `academic_success` uses, applied here to `primary_genre` (182 distinct values) rather than occupation/nationality codes.

```
from recommendation_showcase.collaborative.features import FeatureEngineer, InteractionCountEncoder

X_sample = panel_sample.drop(columns=[config.TARGET_COL])
counted = InteractionCountEncoder().fit_transform(X_sample)
engineered = FeatureEngineer().fit_transform(counted)

fig, axes = plt.subplots(1, 2, figsize=(11, 4))
sns.histplot(engineered["age_clean"].dropna(), bins=40, ax=axes[0])
axes[0].set_title("age_clean (garbage removed)")
sns.histplot(counted["song_popularity"], bins=50, ax=axes[1], log_scale=(False, True))
axes[1].set_title("song_popularity (log y)")
plt.tight_layout()
plt.show()
```



3.3 Methodology

3.3.1 Why one train/holdout split, not k-fold cross-validation

At 7.4M rows, a single fit of a gradient-boosted model already takes real time (verified directly: LightGBM, ~91 seconds for the full join + feature engineering + fit). Unlike Part I, there's no chronological or identity-based leakage risk here that would make a single stratified split misleading — so one honest 80/20 split, not 5-fold CV, is the tractable and still-honest choice.

3.3.2 Model comparison

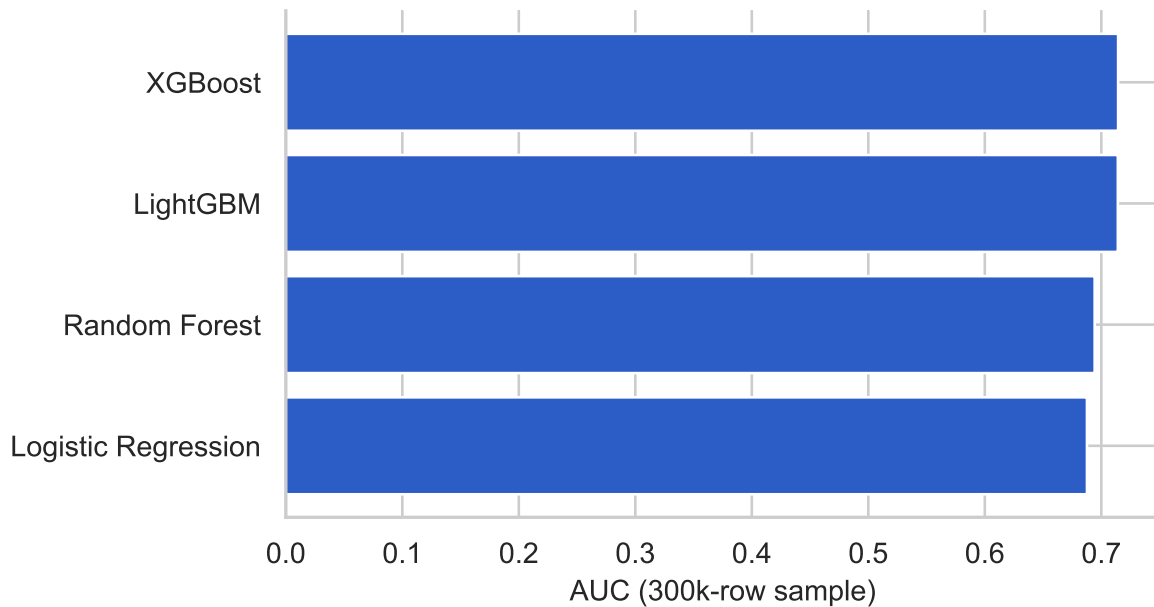
```
from recommendation_showcase.collaborative import model

sweep_sample = data.load_train().sample(300_000, random_state=config.RANDOM_SEED)
sweep_panel = data.merge_side_tables(sweep_sample)

sweep_results = []
for name, factory in model.MODEL_FACTORIES.items():
    result = model.holdout_eval(sweep_panel, estimator=factory())
    sweep_results.append({"model": name, "auc": result["auc"]})
sweep_df = pd.DataFrame(sweep_results).sort_values("auc", ascending=False)
sweep_df
```

	model	auc
3	XGBoost	0.714414
2	LightGBM	0.714181
1	Random Forest	0.694301
0	Logistic Regression	0.687687

```
fig, ax = plt.subplots(figsize=(6, 3.5))
ax.barh(sweep_df["model"], sweep_df["auc"], color=PALETTE[0])
ax.set_xlabel("AUC (300k-row sample)")
plt.gca().invert_yaxis()
plt.show()
```



Random Forest is excluded from `scripts/train_collaborative.py`'s default by design — verified directly that `RandomForestClassifier(n_estimators=200, max_depth=12)` didn't finish fitting on the full 5.9M-row training split even after 500 seconds, while LightGBM finished the equivalent job in 91 seconds. The sweep above uses a 300k-row sample specifically so Random Forest *can* be included for a fair comparison point — not because it would scale to the real dataset.

3.3.3 Full-scale result

```
full_panel = data.load_full_train_panel()
full_result = model.holdout_eval(full_panel, estimator=model.default_estimator())
print(f"LightGBM, full 7.4M-row panel: AUC = {full_result['auc']:.4f}")
print(f"  (trained on {full_result['n_train']:,} rows, evaluated on {full_result['n_holdout']:,} rows)
```

```
LightGBM, full 7.4M-row panel: AUC = 0.7253
  (trained on 5,901,934 rows, evaluated on 1,475,484)
```

3.4 Interpretability

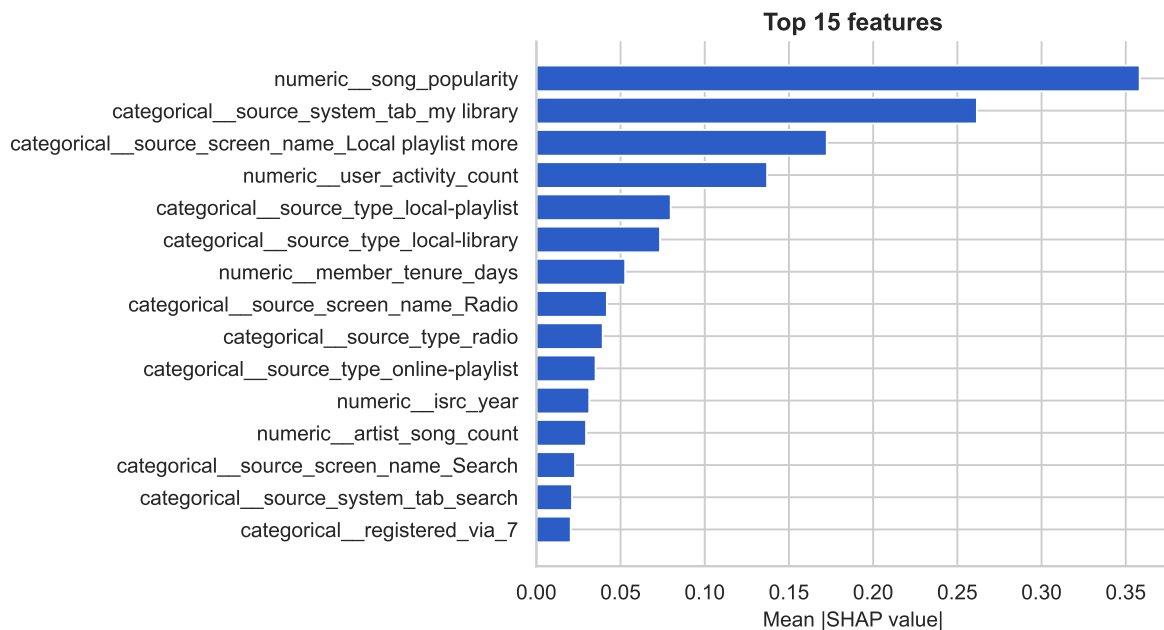
```

from recommendation_showcase.collaborative.interpretability import compute_shap_values, top_

pipeline = model.train_pipeline(sweep_panel, estimator=model.default_estimator())
X_sweep = sweep_panel.drop(columns=[config.TARGET_COL])
explanation, _ = compute_shap_values(pipeline, X_sweep, max_samples=500)
top = top_shap_features(explanation, top_n=15).sort_values("mean_abs_shap")

fig, ax = plt.subplots(figsize=(6, 5))
ax.barh(top["feature"], top["mean_abs_shap"], color=PALETTE[0])
ax.set_xlabel("Mean |SHAP value|")
ax.set_title("Top 15 features")
plt.show()

```



4 Head-to-Head Comparison

Both approaches solve “recommend a song,” but they aren’t interchangeable — the honest comparison is about what each technique needs and what it’s actually good for, not which number is bigger.

```

comparison = pd.DataFrame([
    {

```

```

    "Aspect": "Data required",
    "Content-based (Spotify)": "None - just item features",
    "Collaborative filtering (KKBox)": "Real user/item interaction history",
  },
  {
    "Aspect": "Cold-start (brand-new item/user)",
    "Content-based (Spotify)": "New tracks work immediately (their own features exist)",
    "Collaborative filtering (KKBox)": "New songs/users get 0 interaction counts - a real",
  },
  {
    "Aspect": "Evaluation metric",
    "Content-based (Spotify)": "precision@k (genre-sharing proxy)",
    "Collaborative filtering (KKBox)": "AUC (real Kaggle leaderboard)",
  },
  {
    "Aspect": "Headline result",
    "Content-based (Spotify)": f"{cb_best_precision_exact:.3f} precision@10 (exact genre)",
    "Collaborative filtering (KKBox)": f"{coll_auc:.4f} AUC, full 7.4M-row training set",
  },
  {
    "Aspect": "Explains recommendations by",
    "Content-based (Spotify)": "Feature similarity (radar chart of audio properties)",
    "Collaborative filtering (KKBox)": "Interaction frequency + member/song attributes (S",
  },
  {
    "Aspect": "Dataset scale",
    "Content-based (Spotify)": "~90k tracks (deduplicated)",
    "Collaborative filtering (KKBox)": "7.4M interactions, 2.3M songs, 34k members",
  },
]
comparison

```

	Aspect	Content-based (Spotify)	Collaborative filtering (KKBox)
0	Data required	None — just item features	Real user/item interaction history
1	Cold-start (brand-new item/user)	New tracks work immediately (their own features exist)	New songs/users get 0 interaction counts - a real
2	Evaluation metric	precision@k (genre-sharing proxy)	AUC (real Kaggle leaderboard)
3	Headline result	0.161 precision@10 (exact genre), 0.331 (genre-sharing proxy)	0.7253 AUC, full 7.4M-row training set
4	Explains recommendations by	Feature similarity (radar chart of audio properties)	Interaction frequency + member/song attributes (Spotify)
5	Dataset scale	~90k tracks (deduplicated)	7.4M interactions, 2.3M songs, 34k members

Neither number in the “Headline result” row is comparable to the other in any direct sense —

precision@10 and AUC measure different things on different tasks. What *is* comparable is the discipline behind each: both projects hold out data honestly, compare against a naive baseline before claiming a win, and measure scalability/cost tradeoffs directly rather than assuming them (NCA’s memory limit in Part I; Random Forest’s full-scale timeout in Part II).

5 Discussion

- **Part I’s importance-weighted KNN won** its own comparison, beating unweighted KNN, PCA, and NCA — the two more mathematically sophisticated methods didn’t pay off here. This is a genuine, reportable finding in the same spirit as the CO2 Emissions and academic_success projects’ own honestly-reported negative results (ADASYN, stacking).
- **Part I’s exact-genre precision@k understates recommendation quality, but not by as much as it might look** — the naive baseline’s score grows even faster at the looser genre-family granularity, so the recommender’s real edge is clearest at the strict, exact-genre level.
- **Part II’s real, measured scalability finding:** Random Forest doesn’t finish fitting on the full KKBox training set in a reasonable time, while LightGBM does the same job in under two minutes — worth knowing before assuming a bigger, ostensibly-more-powerful ensemble is always the safer default.
- A limitation worth naming for both: Part I’s genre-sharing is an imperfect proxy for “sounds alike”; Part II’s single train/holdout split (not k-fold CV) is an honest tradeoff for training cost, but one fold’s variance is real.

6 Conclusion

Content-based and collaborative filtering solve the same stated problem — “recommend something” — with almost entirely non-overlapping requirements, evaluation strategies, and failure modes. Building both on real data, under the same standards of honest evaluation and measured (not assumed) engineering tradeoffs, makes the actual difference between the two techniques legible in a way that reading about either one in the abstract doesn’t.